

Operating System & Advanced Mobile Lab 2

Report – Mini Shell Project (sish)

By Sebastien BERTIL SOUCHET (32239360)

Date: 2025/09/30

1. Introduction

This project implements a minimal shell in C called sish.

Its goal is to:

- Read commands typed by the user.
- Parse and tokenize the command line.
- Search for built-in command (exit)
- Search the executable in the directories defined by the PATH environment variable.
- Execute the command in a child process using `execve`.

This shell mimics a very small subset of what Bash or Zsh does. It supports:

- Running executables in \$PATH with args (e.g., `ls -la`, `echo`, `cat`).
- Running executables via absolute/relative paths (e.g., `/bin/ls`, `./a.out`).
- Handling basic errors (command not found, not executable).
- Handling signal (CTRL + C / CTRL + D).

2. Instructions to Build the Program

Requirements

- GCC (or Clang)
- Linux / Unix-like environment

Files

- `main.c` : main shell loop and command handling.
- `sish.h` : header file with macros, constants, and function prototypes.
- `Makefile` : build automation.

Build

Run:

```
make
```

This produces the binary:

```
./sish
```

To clean build files (build folder):

```
make clean
```

To clean everything ((build files & binaries):

```
make fclean
```

To rebuild everything:

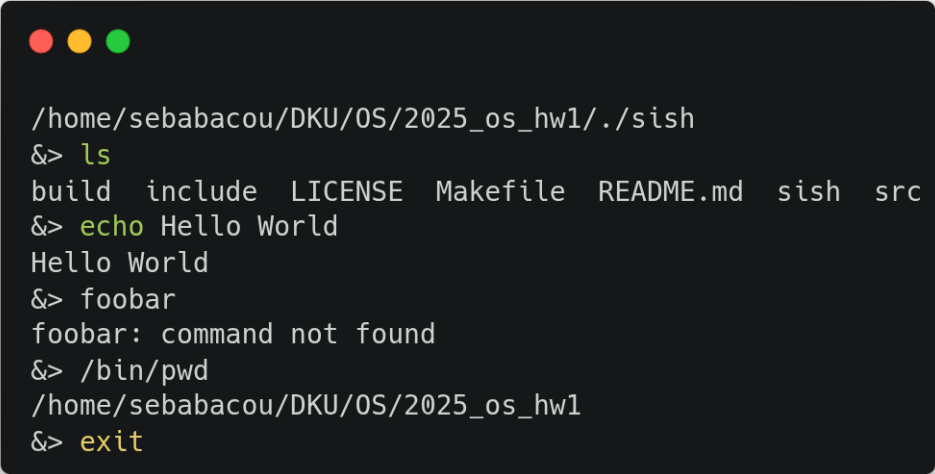
```
make re
```

3. How the Program Works

General Flow:

1. Prompt display: The shell prints `&>` and waits for input.
2. Input reading: The user types a command; input is read with `getline`.
3. Tokenization: The command line is split into tokens (command + arguments).
4. Execution:
 - If the command contains `'/'`, it is executed directly if accessible.
 - Otherwise, the program searches for it in each directory of the `PATH` variable.
 - If found and executable, it forks a child and calls `execve`.
 - If not found, prints an error.

4. Example Usage



```
/home/sebabacou/DKU/OS/2025_os_hw1/./sish
&> ls
build include LICENSE Makefile README.md sish src
&> echo Hello World
Hello World
&> foobar
foobar: command not found
&> /bin/pwd
/home/sebabacou/DKU/OS/2025_os_hw1
&> exit
```

5. Sample Screenshot (terminal capture)

```
~/DKU/OS/2025_os_hw1
(14:24:04 on 32239360_sebastien_BERTIL-SOUCHET • ★)→ ./sish
&> ls -l
total 48
drwxr-xr-x. 2 sebacou sebacou 4096 Sep 30 14:23 build
drwxr-xr-x. 2 sebacou sebacou 4096 Sep 30 13:35 include
-rw-r--r--. 1 sebacou sebacou 1078 Sep 20 19:46 LICENSE
-rw-r--r--. 1 sebacou sebacou 1305 Sep 30 14:01 Makefile
-rw-r--r--. 1 sebacou sebacou 4494 Sep 20 19:46 README.md
-rwxr-xr-x. 1 sebacou sebacou 17784 Sep 30 14:23 sish
drwxr-xr-x. 2 sebacou sebacou 4096 Sep 30 14:21 src
&> echo it work
it work
&> foobar
foobar: command not found
&> exit
~/DKU/OS/2025_os_hw1
(14:24:21 on 32239360_sebastien_BERTIL-SOUCHET • ★)→
```

6. Personal Feedback and Ideas

This project was a good exercise in process management (fork, execve, waitpid) and string manipulation in C.

I noticed how much complexity is hidden behind a real shell. Even simple features like redirection or pipes require major additions (file descriptors, dup2, multiple processes).

Improvements I would like to add:

- Builtins: cd, exit, env.
- Redirections (>, <, >>) to handle files.
- Pipes to chain commands.
- Jobs
- More robust parsing (quotes, escape characters).